Optional word sets (6j, 6k), the disassembler (6l) and SEE (6m)

1. What stage 6j adds

Two assembly-time switches, set like `SERIALPOLL` (default 1, so the default build is unchanged: same size, same words):

Switch	Removes when 0
`STRINGSET`	`/STRING` , `-TRAILING` , `SEARCH` , `BLANK` , `SLITERAL` , and the **headers** of `CMOVE` , `CMOVE>` , `COMPARE`
`STRINGEXT`	`UNESCAPE` , `REPLACES` , `CLR-REPLACES` , `SUBSTITUTE` (code and headers)

Build with, for example, `--define=STRINGSET=0 --define=STRINGEXT=0` .

Layout: the headers are one contiguous run between `>NUMBER` and `S"` , and the code is one contiguous run in section 19. Each switch wraps its run in `IFNE` / `ENDC` . The link of the header after a removed run is set by `H_STRLINK1` and `H_STRLINK2` (`EQU` chosen by the switch), so the dictionary chain stays intact in all four combinations (checked by walking the chain: 269 headers on with everything, 257 with both string switches off; 266 and 254 before the DASM, DIS and SEE headers; the count was 265 and 253, 262 and 250 until stage 6o added (EMIT), (KEY), (KEY?) and IRQ-CHAIN).

2. Dependencies found (the "assuming nothing depends on them" check)

Something did depend on three of them, so they are not fully removable:

- `CMOVE` and `CMOVE>` code is called by `MOVE` (a Core word). Code kept always; only the headers are optional.
- `COMPARE` code is called twice by `ENVIRONMENT?` . Code kept always; only the header is optional.
- `SNAME` (xt to name over all wordlists) sat in the string-extension run

but is not a string word and nothing string-related uses it. It is now always assembled, ahead of the optional runs. The decompiler needs it.

`BLANK` and `SLITERAL` were in other sections (Memory, Compiling). Their code and headers moved into the string runs. Nothing else calls them. Side effect: `WORDS` lists those four words in the string group instead of their old places. The shadow file needs no change (no comment moved or renamed).

3. ROM recovered (measured, either serial build)

Setting	Headers	Code	Total saved
`STRINGEXT=0`	58	599	**657**
`STRINGSET=0`	93	181	**274**
both 0	151	780	**931**

(These are the final figures, including the `ENVIRONMENT?` entries added in 6k. The first measurements in 6j were 641, 262 and 903.)

`BASEDICT` and `BASECODE` are still fixed `EQU` s, so the saved bytes show up as a gap: below `BASECODE` (the dictionary ends early) and above `BASECODEEND` (more padding before `INITCODE`). To use them for something else, lower both constants by the saving.

The `SUBPOOL` RAM ($0200-$023F, 64 bytes) is only used by `REPLACES` . It could also be reclaimed when `STRINGEXT=0` (not done; RAM is not tight).

4. Cautions

- `UNITTESTS=1` stops with an `ERROR` unless both string switches are 1. The check is an `ERROR` directive and an `IFEQ A*B` ; I could only run the default path in my assembler, so please confirm lwasm accepts them.
- `ENVIRONMENT?` (stage 6k): `STRING` and `STRING-EXT` entries exist only when their switch is on, so a missing feature answers false. Cost 28 bytes when both are on.
- ANS test file 27 is split (done): 27 is the String word set, 36 is the String extension. Each uses colon definitions for `S"` , because interpret-mode `S"` shares one buffer.

5. Verification of 6j and 6k

- All four switch combinations assemble in both serial builds; 262 headers with both on, 250 with both off (at 6k, before the 6l and 6m words).
- All 19 unit-test groups pass (group 0 `TSTKEYQ` fails exactly as on 6i in my emulator, which has no pending key).
- ANS 27: 50 run, 0 failed. ANS 36: 14 run, 0 failed (also with `STRINGSET=0`).

6. The disassembler (stage 6l)

Words (switch `DISASM`, default 1; build with `--define=DISASM=0` to drop)

Word	Stack	What it does
`DASM`	`( addr -- addr' )`	Send one instruction as text; return the address of the next one.
`DIS`	`( addr u -- )`	List every instruction that starts in `addr .. addr+u-1`: address, bytes, text, one per line.

Output is hexadecimal only, lwasm style:

```
8000 86 A5           LDA #$A5
8002 96 7B           LDA <$7B
8004 B6 A5 A5        LDA $A5A5
8007 A6 0A           LDA $0A,X
8009 A6 F6           LDA [A,S]
8013 A6 8C 1D        LDA $8033,PCR
8024 8D 10           BSR $8036
801E 34 76           PSHS A,B,X,Y,U
8043 AD 9F 12 34     JSR [$1234]
8070 A8              FCB $A8
```

- Immediate `#$xx`; direct `<$xx`; extended `$xxxx`, or `>$00xx` when the high byte is zero so that it stays extended.
- Indexed: `$0A,X`, `-$0A,X`, `,X`, `A,X`, `B,X`, `D,X`, `,X+`, `,X++`, `,-X`, `,--X`, `[...]` for indirect, `[$xxxx]` for extended indirect. An 8-bit offset that would fit in 5 bits is shown as `<$05,X`, and a 16-bit offset that would fit in 8 bits as `>$0005,X`, so lwasm builds the same bytes.

- PC-relative operands and all branches show the absolute target: `$8033,PCR`, `BRA $8026`. A 16-bit PC-relative offset that would fit in 8 bits is shown as `>$xxxx,PCR`.
- `PSHS` / `PULS` / `PSHU` / `PULU` lists in the order CC A B DP X Y U/S PC. Bit 6 shows `U` for the S-stack ops and `S` for the U-stack ops. An empty list shows `#$00`.
- A byte that is not an instruction shows as `FCB $xx` and is one byte long. That covers unused opcodes, a prefix followed by something that has no prefixed form, a second prefix, undefined indexed postbytes (modes 7, A, E, indirect `,R+` and `,-R`, mode F without indirect) and `TFR` / `EXG` register codes 6, 7, C-F.
- `LSL` / `LSLA` / `LSLB` show as `ASL` / `ASLA` / `ASLB` (same opcode).

Known limits

- Indexed postbyte bits that the CPU ignores (register bits for the PC-relative and extended-indirect forms) are not shown, so such a non-canonical encoding disassembles to the canonical text. 1,083 of the tested encodings are of this kind.
- A 5-bit offset of zero ($00, $20, $40, $60 as postbyte) shows as `$00,X`. lwasm builds `,X` from that text.
- `PSHS #$00` is lwasm syntax as far as I know. My own assembler does not take it.

How it works (about 1.8 KB)

Table driven, not 256 handlers. Opcodes $00-$7F are decoded by high nibble: read-modify-write columns ($0x, $4x-$7x share one 16-entry table), the $1x and $3x tables (name and mode), and the branch names in order for $2x. Opcodes $80-$FF are a 32-entry table (accumulator A side and B side, 16 columns each); the mode comes from bits 4-5 and the immediate width from the column. Prefix pages $10 and $11 only add an 8-entry override list and the long branches. Names live in one pool, the last character of each with bit 7 set. Every character goes out through one routine, `DCH`, which is silenced by a flag so `DIS` can run each instruction once to find its length before it prints the bytes.
The tables are generated by `gen_dis_tables.py` (in the project) and pasted into the source between `DNAMES:` and `DIXT:`.

ROM cost, measured

Part	Bytes
Code	1,231
Name pool and tables	541
Headers `DASM`, `DIS`	17
19 bytes of direct-page globals	0 ROM
Total	**1,789**

`BASECODE` moved from $D936 to $D240 and `BASEDICT` from $CF87 to $C880 (`DISASM=1`). With `DISASM=0` the constants are the old ones and the image is **byte for byte identical to 6k** (checked in both serial builds).
Spare ROM above the code: polled 333 bytes, interrupt 14 bytes.

Unit-test regions

With `DISASM=1` the region below `BASEDICT` is $C880-$C100 = 1,920 bytes.
Groups that fit and pass with `DISASM=1`, `DECOMP=0`: 0, 2, 13, 14, 15, 17 (16 until stage 6n added ENVIRONMENT? unit tests; 5 until stage 6o added the vectored I/O and the interrupt chain). With both switches on no group fits since stage 6o (13 and 15 did before). With `DISASM=0` all 22 groups fit; group 10 was split into 10 and 21 in stage 6o.
The other groups need `DISASM=0`. When `UNITTESTS` is defined and nonzero and `DISASM` is not given, `DISASM` defaults to 0, so the usual `--define=UNITTESTS=1 --define=TSTSELECTOR=n` command works unchanged.

Verification

- Round trip: 20,432 byte sequences were disassembled by `DASM` in the emulator, the text reassembled with my assembler at the same address, and the bytes compared. Covered: every opcode on all three pages with three operand patterns, all 256 postbytes for every indexed opcode, all 256 postbytes for `TFR` / `EXG`, extra offset sizes (+/-, 5, 8 and 16 bit) for the indexed forms, and double prefixes. Whether a byte is legal comes from my opcode tables, not from `DASM`. Result: 0 differences, except the known limits above (4 empty `PSHS` lists that my assembler rejects, and 1,083 non-canonical postbytes that cannot round trip).
- `37_disassembler.tests.fs`: 64 tests, 0 failed. Each checks the length `DASM` returns (instruction length, or 1 for a non-instruction) for an encoding. The text is printed to the terminal as the tests run. The text

itself is checked by the round trip above, not by this file.

- All 19 unit-test groups pass with `DISASM=0`; 8 groups pass with `DISASM=1`. ANS files 27 (50/0), 33 (76/0), 34 (15/0), 36 (14/0) pass.
- Not run: lwasm, MAME, hardware.

7. SEE, the decompiler (stage 6m)

The source has a SECTION 28 banner (disassembler and decompiler) ahead of the `DISASM` block. The main document lists it as Section 8.29.

Word (switch `DECOMP`, default follows `DISASM`)

`SEE ( "name" -- )` shows a definition as Forth source. `DECOMP=1` needs `DISASM=1` (SEE sends machine code through `DASM` and its output through the same routine); `DECOMP=1` with `DISASM=0` stops with an `ERROR`. `--define=DECOMP=0` keeps the disassembler and drops SEE; the image is then byte for byte the 6l image.

```
: A2 DUP 0= IF DROP 1 ELSE DUP 1- A2 * THEN ;
: A4 BEGIN DUP 5 < WHILE 1+ REPEAT ;
: A7 ." hello" S" abc" TYPE ABORT" bad" ;
: B1 5 0 DO 3 0 DO I J + . LOOP LOOP ;
CREATE M7 DATA 28753 DOES> @ 1+ ;
CODE DROP
LEAU $02,U
RTS
```

- **How:** a colon definition is a string of `JSR xt` instructions. A few run-time words are followed by an inline cell (`LIT`, `BRANCH`, `ZBRANCH`, `DOTEST`, `DOPLUSTEST`, `QDOSETUP`, `EXITUNLOOP`) or a counted string (`DOSTR`, `DOTSTR`, `DOCSTR`, `DOABORTQUOTE`). SEE steps over them using a table (`DKT`) and names every other `JSR` target with `SNAME`.
- **Control structures** come from the branch targets. Each time SEE reaches an address it scans the definition for branches that end or start there: `THEN` and `BEGIN` are printed before the instruction, and `ZBRANCH` / `BRANCH` become `IF`, `ELSE`, `UNTIL`, `AGAIN`, `WHILE` or `REPEAT` by direction and by what branches to what. `DO`, `?DO`, `LOOP`, `+LOOP`, `EXIT`, strings and `DOES>` are direct.
- **Literals** are shown in the current `BASE` (signed), so `HEX SEE x`

shows hex.

- **Machine code words** (first byte is not a `JSR` or `RTS`) are shown as `CODE name` and one `DASM` line per instruction, up to the first `RTS`. A machine-code fragment inside a colon definition is also shown with `DASM`.
- **Words made by CREATE, VARIABLE, CONSTANT, 2VARIABLE, BUFFER:** shown as `CREATE name DATA <address>`. If the behaviour is not the default, it follows `DOES>`: its name if it has one (a CONSTANT shows `@`), or the decompiled code of the `DOES>` part.

Known limits

- `CASE` shows as the code it compiles: `OVER = IF DROP ... ELSE ... THEN`. `OF`, `ENDOF` and `ENDCASE` are not rebuilt.
- An `AGAIN` loop that sits at the end of an `IF` branch (`x IF BEGIN y AGAIN THEN`) shows as `REPEAT`; the shape is otherwise right. Loops at the end of an `IF` branch that end with `REPEAT` (`x IF BEGIN y WHILE z REPEAT THEN`) show correctly.
- A word with no header (`:NONAME`) that is called from a definition shows as the `DASM` text `JSR $xxxx`.
- SEE stops after 255 items.
- Names are the dictionary names, so a word that was `RECURSE`d shows its own name.
- The output is the code, not the original source: comments, `[CHAR]` and `[']` (shown as a number), `POSTPONE` and numbers typed in another base are not recoverable.

ROM cost, measured

Part	Bytes
`DSTEP`, `DQ`, `DMARK`, handlers, `SEE`	797
Kind table and word list (`DKT`, `DKEYS`)	96
Header `SEE`	8
21 bytes of direct-page globals	0 ROM
Total	**901**

With both switches on, `BASECODE` is $CEC0 and `BASEDICT` is $C4F8 (was $D240 and $C880 with the disassembler alone). Spare ROM above the code: polled 336 bytes, interrupt 17 bytes. The region below `BASEDICT` is 1,016 bytes, so with both on only unit-test groups 13 and 15 fit (both pass). With `DECOMP=0` the 6l list applies (groups 0, 2, 5, 13, 14, 15, 17 since stage 6n). The other groups need `DISASM=0`, which is the default whenever `UNITTESTS` is nonzero.

DISASM	DECOMP	BASECODE	BASEDICT	Added ROM
0	0	$D936	$CF87	0 (identical to 6k)
1	0	$D240	$C880	1,789
1	1	$CEC0	$C4F8	2,690

Verification

- `see_check.py` (in the project): defines 19 words, runs SEE and compares the text exactly: nested IF/ELSE, BEGIN/UNTIL/WHILE/REPEAT/AGAIN, DO/LOOP/?DO/+LOOP with I and J, strings, empty `S" "`, EXIT, CASE, VARIABLE, CONSTANT and a DOES> word. 0 differences.
- Also run and checked by eye: SEE of primitives (`DROP`, `1+`, `CELLS`, `ABS`, `.`) and of SEE itself (a colon word with machine code in it).
- Everything from 6l still passes on 6m: the 20,432-sequence round trip (same 4 known cases), 64 DASM length tests, all 19 unit-test groups (`DISASM=0` default), ANS 27, 33, 34, 36.
- Both switch states give the images described above (`DISASM=0` identical to 6k, `DECOMP=0` identical to 6l).
- Not run: lwasm on 6m, MAME, hardware. The `ERROR` for `DECOMP=1` with `DISASM=0` is the same directive lwasm already accepted for the string guard.